```cpp
//! Arrays
/*
    ! it is a  drived data type that is used to store a collection of elements of the same type.
    ~ it is a type of data structure that allows you to store a
    ~ collection of elements of the same type stored in contiguous memory locations.

    * Arrays
        ! is a variable that can hold/ store multiple values
        ! of the same type.
    * for example : string is an array that can hold multiple/stores characters.
    # you can access the elements of an array using the index of the element.
    # the index of the first element is 0 and the index of the last element is
    // ex :
        string name = "John";
        cout << name[0] << endl; // output : J
        cout << name[1] << endl; // output : o
        cout << name[2] << endl; // output : h
        cout << name[3] << endl; // output : n
        cout << name[4] << endl; // output : (null character) because
        the string is terminated with a null character '\0' at the end of the string.
    You, 3 days ago • philo
    ! syntax of array declaration :
    # data_type array_name[array_size];
        array_name : is the name of the array.
        data_type : is the type of the elements that will be stored in the array.
        array_size : is the number of elements that will be stored in the array.
    ! example of array declaration :
    # int numbers[5]; // this is an array of integers that can store 5
    # double prices[10]; // this is an array of doubles that can store 10
    # char letters[26]; // this is an array of characters that can store 26
    # string names[3]; // this is an array of strings that can store 3

    ! you can also initialize an array at the time of declaration :
    // this is an array of integers that is initialized with the values 1, 2, 3, 4, 5
        int numbers[5] = {1, 2, 3, 4, 5};
    // this is an array of doubles that is initialized with the
    // values 9.99, 19.99, 29.99, 39.99, 49.99, 59.99, 69.99, 79.99, 89.99, 99.99
        double prices[10] = {9.99, 19.99, 29.99, 39.99, 49.99, 59.99, 69.99, 79.99, 89.99, 99.99};
    // this is an array of characters that is initialized with the
    // values A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z
        char letters[26] = {'A', 'B', 'C', 'D ', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q',
        'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z'};
     // this is an array of strings that is initialized with the values "John", "Jane", "Doe"
        string names[3] = {"John", "Jane", "Doe"};

*/
```

```cpp
//# ex
#include <iostream>
using namespace std;
int main()
{
    // declare an array of integers that can store 5 elements
    int numbers[5] = {1, 2, 3, 4, 5}; // initialize the array with the values 1, 2, 3, 4, 5

    // access the elements of the array using the index
    cout << "The first element of the array is: " << numbers[0] <<
        endl; // output : 1
    cout << "The second element of the array is: " << numbers[1] <<
        endl; // output : 2
    cout << "The third element of the array is: " << numbers[2] <<
        endl; // output : 3
    cout << "The fourth element of the array is: " << numbers[3] <<
        endl; // output : 4
    cout << "The fifth element of the array is: " << numbers[4] <<
        endl; // output : 5

    ////////////////////////////
    int numbers2[5]; // declare an array of integers that can store 5 elements
    // initialize the array with the values 10, 20, 30, 40, 50
    numbers2[0] = 10;
    numbers2[1] = 20;
    numbers2[2] = 30;
    numbers2[3] = 40;
    numbers2[4] = 50;

    return 0;
}
```

FUNCTION LINES : 25

```cpp
////////////////////////////////////////////////////////
```

```cpp
//! Arrays with functions
/*
! important note about arrays and functions :
        $  the array pass to the functions by reference, which means
                that the function can modify the original array.
        $ it opposite the structures  that pass by value, which means
                        that the function cannot modify the original structure
        % so just pass the name of the array to the function without the square brackets and the size of the array,
                or (&) before the name of the array,
                and the function will be able to access and modify the original array.
! note :
        ~ when you pass an array to a function, you are actually passing
                a pointer to the first element of the array.
        this means that any changes made to the array inside the function
                will affect the original array outside the function.


*/
```

```cpp
//# ex
#include <iostream>
using namespace std;

void ResetArray(int arr[3])  You, 2 hours ago • minishell
{
        cout << "please enter number 1 : " << endl;
        cin >> arr[0];
        cout << "please enter number 2 : " << endl;
        cin >> arr[1];
        cout << "please enter number 3 : " << endl;
        cin >> arr[2];
}
—— FUNCTION LINES : 6 ——

void PrintArray(int arr[3])
{
        cout << "the first element of the array is : " << arr[0] << endl;
        cout << "the second element of the array is : " << arr[1] << endl;
        cout << "the third element of the array is : " << arr[2] << endl;
}
—— FUNCTION LINES : 3 ——


int main ()  You, 2 hours ago • minishell
{
        int arr[3];
        ResetArray(arr); // pass the array to the function
        PrintArray(arr); // pass the array to the function
        return 0;
}

////////////////////////////////////////////////////////////////////////////////

//! Arrays of structures

//# ex
#include <iostream>
using namespace std;

You, 8 minutes ago | 1 author (You)
struct Student
{
        string name;
        int age;
        double grade;
};

```

```cpp
int main ()
{
        Student students[3]; // declare an array of structures that can store 3 elements

        // initialize the array of structures with the values
        students[0].name = "John";
        students[0].age = 20;
        students[0].grade = 85.5;
        You, 9 minutes ago • Uncommitted changes
        students[1].name = "Jane";
        students[1].age = 21;
        students[1].grade = 90.0;

        students[2].name = "Doe";
        students[2].age = 22;
        students[2].grade = 95.0;

        // access the elements of the array of structures using the index
        cout << "The first student name is : " << students[0].name << endl;
        cout << "The first student age is : " << students[0].age << endl;
        cout << "The first student grade is : " << students[0].grade << endl;

        cout << "The second student name is : " << students[1].name << endl;
        cout << "The second student age is : " << students[1].age << endl;
        cout << "The second student grade is : " << students[1].grade << endl;

            cout << "The third student name is : " << students[2].name << endl;
            cout << "The third student age is : " << students[2].age << endl;
            cout << "The third student grade is : " << students[2].grade << endl;

            return 0;
    }
```